```
1 : // Implementation of Singly Linked List using C
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

// Traverse
void traverse(struct Node* head) {
   struct Node* temp = head;

   while (temp != NULL) {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }

   printf("NULL\n");
}

// Insert at end
void insertAtEnd(struct Node** head, int value) {

   struct Node* newNode =
      (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}

// Insert at beginning
void insertAtBeginning(struct Node** head, int value) {

    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = *head;
```

```
    *head = newNode;
}

// Insert at position
void insertAtPosition(struct Node** head, int value, int pos) {

    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    // Insert at first position
    if (pos == 1) {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    struct Node* temp = *head;

    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
```

```
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

// Delete from beginning
void deleteFromBeginning(struct Node** head) {

    if (*head == NULL) {
        return;
    }

    struct Node* temp = *head;

    *head = (*head)->next;

    free(temp);
}

// Delete from end
void deleteFromEnd(struct Node** head) {

    if (*head == NULL) {
        return;
```

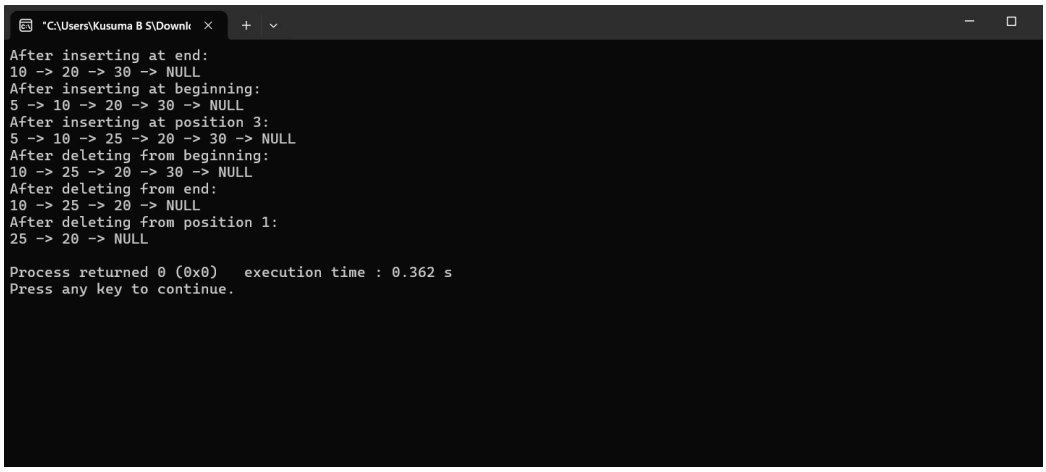
```
    }

    struct Node* temp = *head;

    // Only one node
    if (temp->next == NULL) {
        free(temp);
        *head = NULL;
        return;
    }

    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;
}

// Delete from position
void deleteFromPosition(struct Node** head, int pos) {

    if (*head == NULL) {
        return;
    }
```

```
    struct Node* temp = *head;

    // Delete first node
    if (pos == 1) {
        *head = temp->next;
        free(temp);
        return;
    }

    for (int i = 1; i < pos - 1 && temp->next != NULL; i++) {
        temp = temp->next;
    }

    if (temp->next == NULL) {
        return;
    }

    struct Node* delNode = temp->next;

    temp->next = delNode->next;

    free(delNode);
}

// Main function
int main() {
```

```
struct Node* head = NULL;

insertAtEnd(&head, 10);
insertAtEnd(&head, 20);
insertAtEnd(&head, 30);

printf("After inserting at end:\n");
traverse(head);

insertAtBeginning(&head, 5);

printf("After inserting at beginning:\n");
traverse(head);

insertAtPosition(&head, 25, 3);

printf("After inserting at position 3:\n");
traverse(head);

deleteFromBeginning(&head);

printf("After deleting from beginning:\n");
traverse(head);

deleteFromEnd(&head);
```

```
    printf("After deleting from end:\n");
    traverse(head);

    deleteFromPosition(&head, 1);

    printf("After deleting from position 1:\n");
    traverse(head);

    return 0;
}
```

Output :



2 : // Sparse Matrix Representation and Transpose using Triplet Form in C

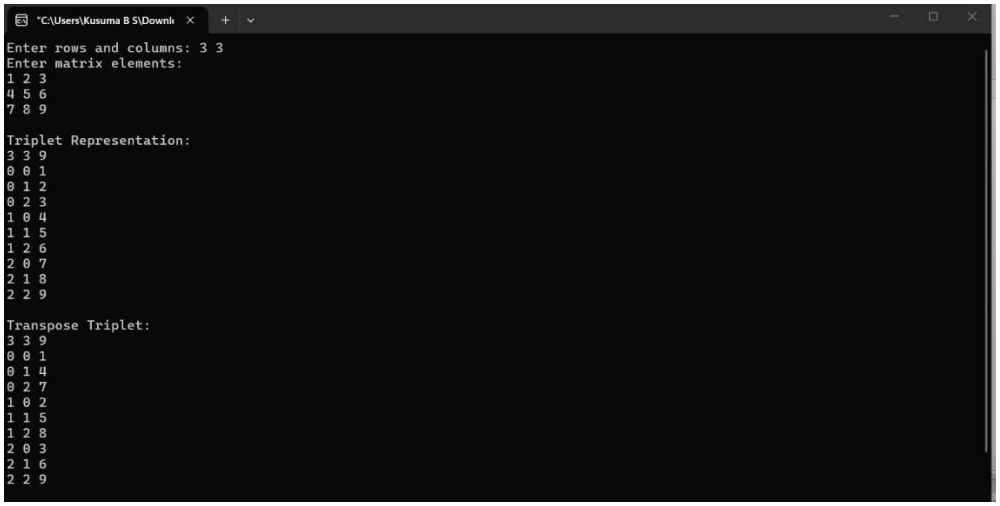
```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int matrix[10][10], triplet[50][3], transpose[50][3];
```

```
int i, j, m, n, k = 1;

printf("Enter rows and columns: ");
scanf("%d %d", &m, &n);

printf("Enter matrix elements:\n");

for(i = 0; i < m; i++) {
    for(j = 0; j < n; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

// Convert to triplet form
triplet[0][0] = m;
triplet[0][1] = n;
triplet[0][2] = 0;

for(i = 0; i < m; i++) {
    for(j = 0; j < n; j++) {

        if(matrix[i][j] != 0) {

            triplet[k][0] = i;
            triplet[k][1] = j;
            triplet[k][2] = matrix[i][j];
```

```
            k++;
            triplet[0][2]++;
        }
    }
}

printf("\nTriplet Representation:\n");

for(i = 0; i < k; i++) {
    printf("%d %d %d\n",
            triplet[i][0],
            triplet[i][1],
            triplet[i][2]);
}

// Transpose of triplet
transpose[0][0] = triplet[0][1];
transpose[0][1] = triplet[0][0];
transpose[0][2] = triplet[0][2];

k = 1;

for(i = 0; i < n; i++) {

    for(j = 1; j <= triplet[0][2]; j++) {
```

```
            if(triplet[j][1] == i) {

                transpose[k][0] = triplet[j][1];
                transpose[k][1] = triplet[j][0];
                transpose[k][2] = triplet[j][2];

                k++;
            }
        }
    }

    printf("\nTranspose Triplet:\n");

    for(i = 0; i < k; i++) {

        printf("%d %d %d\n",
                transpose[i][0],
                transpose[i][1],
                transpose[i][2]);
    }

    return 0;
}
```

Output :



3: // Implementation of Array Operations using C

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int arr[100], n, i, pos, value, choice, found;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    do {
```

```
printf("\n--- MENU ---\n");
printf("1. Traversal\n");
printf("2. Insertion\n");
printf("3. Deletion\n");
printf("4. Searching\n");
printf("5. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice) {

    case 1:

        printf("Array elements are:\n");

        for(i = 0; i < n; i++) {
            printf("%d ", arr[i]);
        }

        printf("\n");
        break;

    case 2:
```

```
    printf("Enter position and value: ");
    scanf("%d %d", &pos, &value);

    for(i = n; i >= pos; i--) {
        arr[i] = arr[i - 1];
    }

    arr[pos - 1] = value;
    n++;

    printf("Element inserted successfully.\n");
    break;

case 3:

    printf("Enter position to delete: ");
    scanf("%d", &pos);

    for(i = pos - 1; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }

    n--;

    printf("Element deleted successfully.\n");
    break;
```

```
case 4:

    printf("Enter element to search: ");
    scanf("%d", &value);

    found = 0;

    for(i = 0; i < n; i++) {

        if(arr[i] == value) {

            printf("Element found at position %d\n", i + 1);

            found = 1;
            break;
        }
    }

    if(found == 0) {
        printf("Element not found\n");
    }

    break;

case 5:
```

```
            printf("Exiting...\n");
            break;

        default:

            printf("Invalid choice\n");
        }

    } while(choice != 5);

    return 0;
}
```

Output :

4: Implementation of Stack using Array

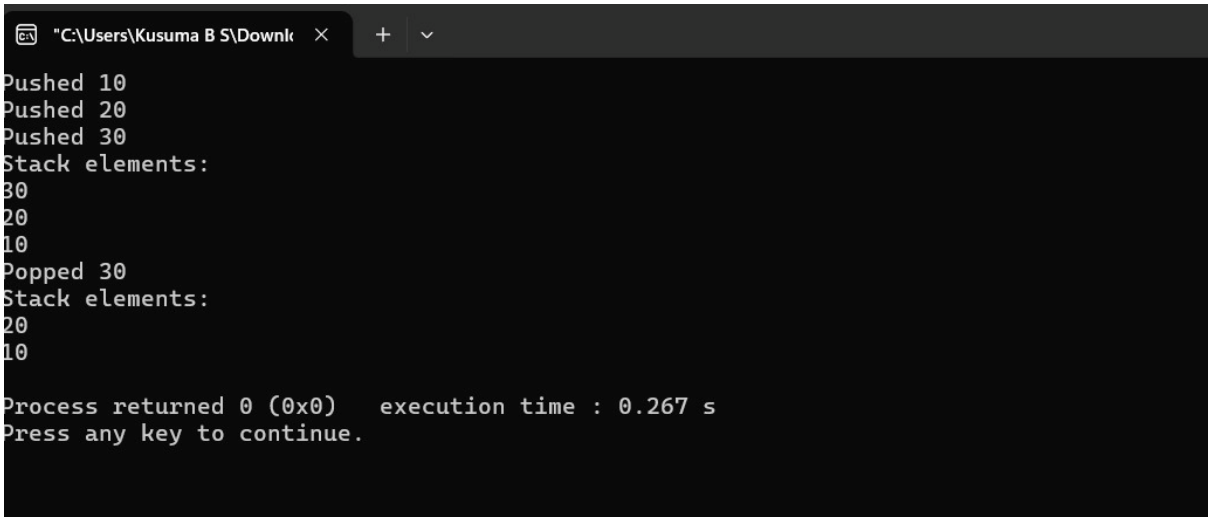
```
#include <stdio.h>

#define MAX 5
```

```
int stack[MAX];
int top = -1;

// Push
void push(int value) {

    if (top == MAX - 1)
        printf("Stack Overflow\n");

    else {
        top++;
        stack[top] = value;

        printf("Pushed %d\n", value);
    }
}

// Pop
void pop() {

    if (top == -1)
        printf("Stack Underflow\n");

    else {

        printf("Popped %d\n", stack[top]);
```

```
        top--;
    }
}

// Display
void display() {

    if (top == -1)
        printf("Stack is empty\n");

    else {

        printf("Stack elements:\n");

        for (int i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

// Main
int main() {

    push(10);
    push(20);
    push(30);
```

```
    display();

    pop();

    display();

    return 0;
}
```

Output :



5 : Infix to Postfix Conversion using Stack in C

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>


#define MAX 100
```

```
char stack[MAX];
int top = -1;

// Push
void push(char c) {

    if (top >= MAX - 1) {

        printf("Stack Overflow\n");
        return;
    }

    stack[++top] = c;
}

// Pop
char pop() {

    if (top < 0) {
        return '\0';
    }

    return stack[top--];
}

// Peek
```

```
char peek() {

    if (top < 0) {
        return '\0';
    }

    return stack[top];
}

// Precedence
int precedence(char op) {

    switch(op) {

        case '+':
        case '-':
            return 1;

        case '*':
        case '/':
            return 2;

        case '^':
            return 3;

        default:
```

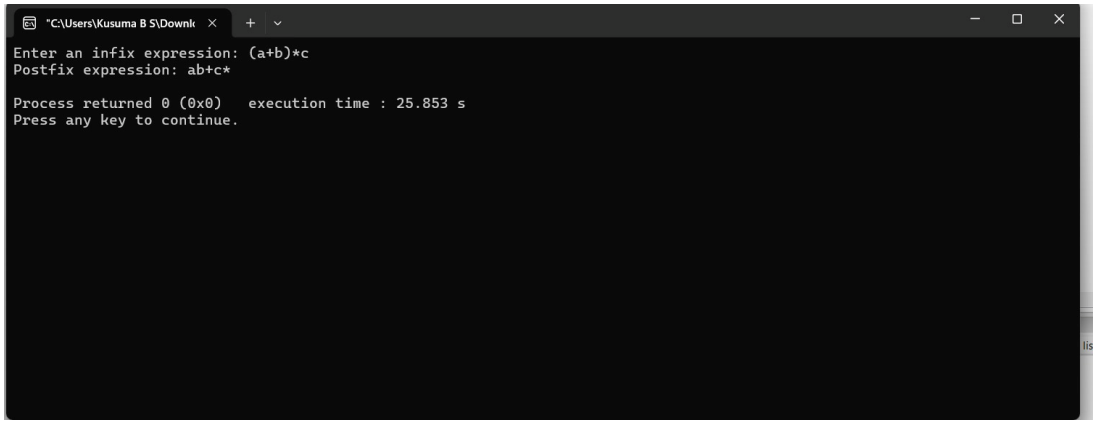
```
        return 0;
    }
}

// Check operator
int isOperator(char c) {

    return (c == '+' ||
        c == '-' ||
        c == '*' ||
        c == '/' ||
        c == '^');
}

// Infix to Postfix
void infixToPostfix(char *infix, char *postfix) {

    int i = 0, j = 0;
    char c;

    while ((c = infix[i++]) != '\0') {

        // Operand
        if (isalnum(c)) {

            postfix[j++] = c;
```

```
}

// Left parenthesis
else if (c == '(') {

    push(c);
}

// Right parenthesis
else if (c == ')') {

    while (top >= 0 && peek() != '(') {

        postfix[j++] = pop();
    }

    pop();
}

// Operator
else if (isOperator(c)) {

    if (c == '^') {

        while (top >= 0 &&
                precedence(peek()) > precedence(c)) {
```

```
                postfix[j++] = pop();
            }
        }

        else {

            while (top >= 0 &&
                precedence(peek()) >= precedence(c)) {

                postfix[j++] = pop();
            }
        }

        push(c);
    }
  }

  // Remaining operators
  while (top >= 0) {

    postfix[j++] = pop();
  }

  postfix[j] = '\0';
}
```

```
// Main
int main() {

    char infix[MAX], postfix[MAX];

    printf("Enter an infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

Output:



6: “Postfix Expression Evaluation using Stack in C”

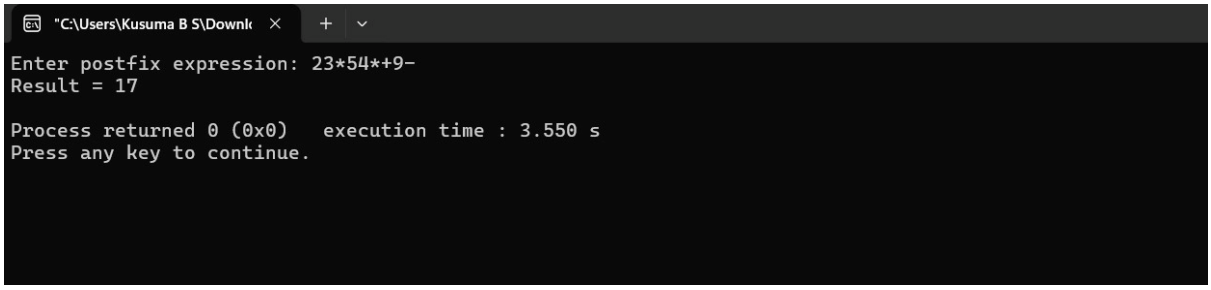
```
#include <stdio.h>
#include <ctype.h>
```

```
#define MAX 100

int stack[MAX];
int top = -1;

// Push operation
void push(int value)
{
    top++;
    stack[top] = value;
}

// Pop operation
int pop()
{
    return stack[top--];
}

// Main function
int main()
{
    char postfix[MAX];
    int i, op1, op2, result;

    printf("Enter postfix expression: ");
    scanf("%s", postfix);
```

```
for(i = 0; postfix[i] != '\0'; i++)
{
    // If operand
    if(isdigit(postfix[i]))
    {
        push(postfix[i] - '0');
    }

    // If operator
    else
    {
        op2 = pop();
        op1 = pop();

        switch(postfix[i])
        {
            case '+':
                result = op1 + op2;
                break;

            case '-':
                result = op1 - op2;
                break;

            case '*':
```

```
                result = op1 * op2;
                break;

            case '/':
                result = op1 / op2;
                break;

            case '%':
                result = op1 % op2;
                break;

            default:
                printf("Invalid operator\n");
                return 1;
        }

        push(result);
    }
  }

  printf("Result = %d\n", pop());

  return 0;
}
```

Output :



7: a : “Implementation of Queue using Array”

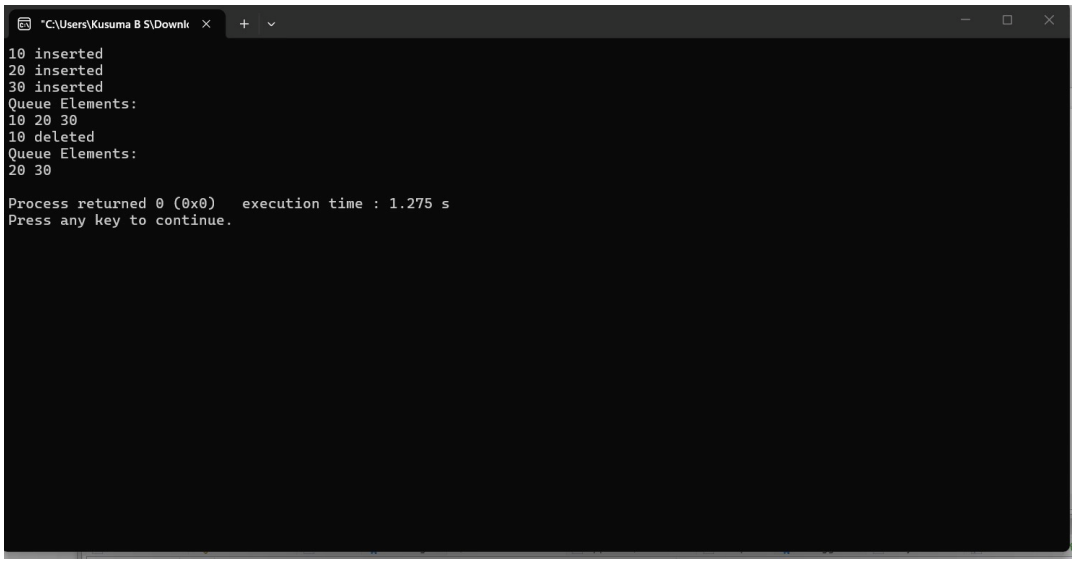
```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Insert element
void enqueue(int value)
{
   if(rear == MAX - 1)
   {
      printf("Queue Overflow\n");
   }

   else
   {
      if(front == -1)
      {
```

```
        front = 0;
    }

    rear++;

    queue[rear] = value;

    printf("%d inserted\n", value);
  }
}

// Delete element
void dequeue()
{
  if(front == -1 || front > rear)
  {
    printf("Queue Underflow\n");
  }

  else
  {
    printf("%d deleted\n", queue[front]);

    front++;
  }
}
```

```
// Display queue
void display()
{
    int i;

    if(front == -1 || front > rear)
    {
        printf("Queue is Empty\n");
    }

    else
    {
        printf("Queue Elements:\n");

        for(i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }

        printf("\n");
    }
}

// Main function
int main()
```

```
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}
```

Output :



7: b: Queue Operations using Linked List in C

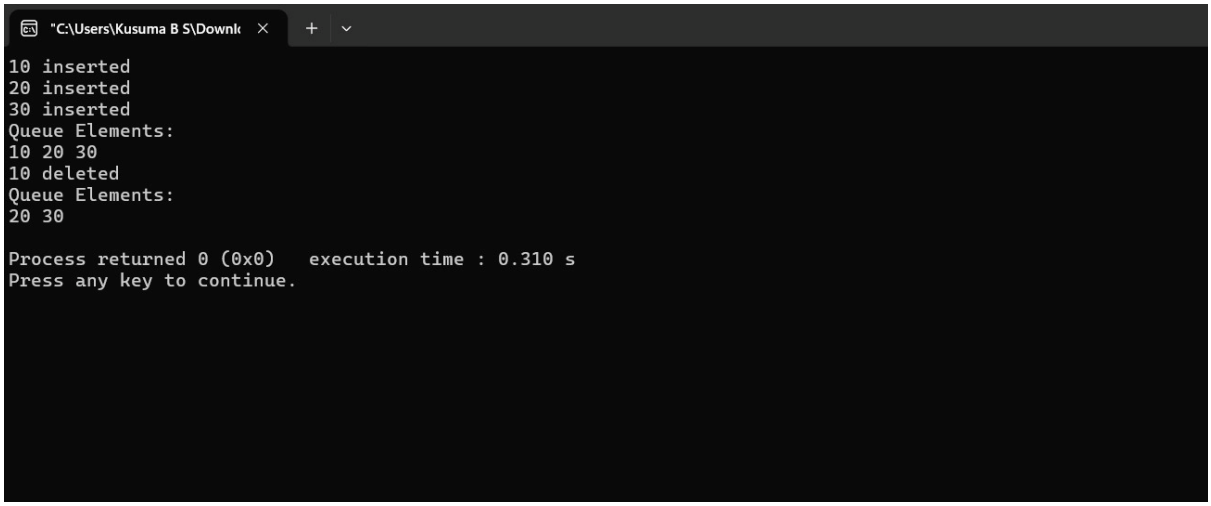
```
#include <stdio.h>
#include <stdlib.h>
```

```
// Node structure
struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

// Enqueue operation
void enqueue(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if(newNode == NULL)
    {
        printf("Memory allocation failed\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;
```

```
    // If queue is empty
    if(rear == NULL)
    {
        front = rear = newNode;
    }

    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted\n", value);
}

// Dequeue operation
void dequeue()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
    }
```

```
    else
    {
        temp = front;

        printf("%d deleted\n", front->data);

        front = front->next;

        // If queue becomes empty
        if(front == NULL)
        {
            rear = NULL;
        }

        free(temp);
    }
}

// Display queue
void display()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue is Empty\n");
```

```
    }

    else
    {
        temp = front;

        printf("Queue Elements:\n");

        while(temp != NULL)
        {
            printf("%d ", temp->data);

            temp = temp->next;
        }

        printf("\n");
    }
}

// Main function
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
```

```
    display();

    dequeue();

    display();

    return 0;
}
```

Output :